ГУАП

КАФЕДРА № 41

ОТЧЕТ

ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А.С. Распкопина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ

ИСПОЛЬЗОВАНИЕ НЕЙРОННЫХ СЕТЕЙ ПРЯМОГО РАСПРОСТРАНЕНИЯ ДЛЯ РЕШЕНИЯ ЗАДАЧ КЛАССИФИКАЦИИ

по курсу:

МАШИННОЕ ОБУЧЕНИЕ И БОЛЬШИЕ ДАННЫЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4316		М.С. Пронь
		подпись, дата	инициалы, фамилия

Санкт-Петербург, 2026

СОДЕРЖАНИЕ

1 Цель работы 3

2 Задачи 3

3 Индивидуальный задание 3

4 Ход работы 3

4.1 Импорт библиотек 3

4.2 Загрузка и первичный анализ данных 4

4.3 Предобработка и анализ данных 5

4.4 Построение архитектуры нейронной сети 10

4.5 Обучение модели 11

4.6 Оценка качества модели 13

4.7 Дополнительное исследование: сравнение оптимизаторов 14

5 Выводы 17

1 Цель работы

Изучение основ работы с нейронными сетями прямого распространения (FNN) для классификации данных, обучение модели на подготовленном датасете, анализ и оценка полученных результатов.

2 Задачи

1) Ознакомиться с принципом работы сети прямого распространения (FNN) и её применением в задачах классификации.
2) Подготовить датасет для обучения модели.
3) Реализовать и обучить нейронную сеть прямого распространения (FNN) с использованием выбранного инструмента (PyTorch, TensorFlow или Keras).
4) Провести обучение сети на подготовленных данных.
5) Оценить точность работы модели и проанализировать полученные результаты.
6) Составить отчет, в котором будет описан процесс работы и выводы.

3 Индивидуальный задание

Вариант 20: Используется датасет Car Evaluation, который содержит информацию о классах автомобилей на основе различных характеристик. В рамках дополнительного исследования необходимо сравнить три оптимизатора: AdamW, Adam и Adagrad при фиксированных значениях learning rate и batch size.

4 Ход работы

4.1 Импорт библиотек

Были импортированы библиотеки для работы с нейронными сетями (PyTorch, модули torch.nn и torch.optim), обработки данных (numpy, pandas), визуализации (matplotlib, seaborn) и предобработки (sklearn). Для воспроизводимости результатов были зафиксированы генераторы случайных чисел со значением 42.

Листинг 1 – Импорт необходимых библиотек для работы

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

torch.manual_seed(42)
np.random.seed(42)
```

4.2 Загрузка и первичный анализ данных

Датасет Car Evaluation содержит информацию об автомобилях с шестью категориальными признаками: цена покупки (buying), стоимость обслуживания (maint), количество дверей (doors), вместимость пассажиров (persons), размер багажника (lug_boot) и уровень безопасности (safety). Целевая переменная (class) принимала четыре значения: unacc (неприемлемый), acc (приемлемый), good (хороший) и vgood (очень хороший). Все признаки являлись категориальными, поэтому требовалось числовое кодирование. Файл не содержал заголовков, поэтому названия столбцов были заданы вручную при загрузке.

Листинг 2 – Загрузка датасета с указанием названий столбцов

```
columns = ['buying', 'maint', 'doors', 'persons', 'lug_boot',
'safety', 'class']
data = pd.read_csv('https://drive.google.com/uc?export=download&
id=...',
                   names=columns)
data.head()
```

Результат загрузки представлен на рисунке 1.

	buying	maint	doors	persons	lug_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

Рисунок 1 – Изначальный датасет

Метод `.info()` выводил типы данных, количество непустых значений и объем памяти (рис. 2).

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1728 entries, 0 to 1727
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   buying    1728 non-null   object
 1   maint     1728 non-null   object
 2   doors     1728 non-null   object
 3   persons   1728 non-null   object
 4   lug_boot  1728 non-null   object
 5   safety    1728 non-null   object
 6   class     1728 non-null   object
dtypes: object(7)
memory usage: 94.6+ KB
```

Рисунок 2 – Информация о датасете

4.3 Предобработка и анализ данных

Проверка качества данных включала поиск пропущенных значений методом `.isna().sum()`, явных дубликатов методом `.duplicated().sum()` и вывод уникальных значений для выявления неявных дубликатов.

Листинг 3 – Проверка качества данных: пропуски и дубликаты

```
print(data.isna().sum())
print('Количество явных дубликатов:', data.duplicated().sum())

for col in data.columns:
    print(f'{col}: {list(data[col].unique())}')
```

Датасет был полным, не содержал пропусков и дубликатов, был готов к обработке. Для анализа распределения целевой переменной была построена столбчатая диаграмма, так как несбалансированность классов влияет на качество обучения.

Листинг 4 – Визуализация распределения целевой переменной

```
plt.figure(figsize=(7, 4))
data['class'].value_counts().plot(
    kind='bar', color=['steelblue', 'orange', 'green', 'red'],
    edgecolor='black'
)
plt.title('Распределение классов автомобилей')
plt.xlabel('Класс')
plt.ylabel('Количество записей')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Из графика (рис. 3) видно, что датасет несбалансированный: класс unacc значительно преобладает над остальными, а классы good и vgood представлены в значительно меньшем количестве. Это стоит учитывать при анализе результатов.

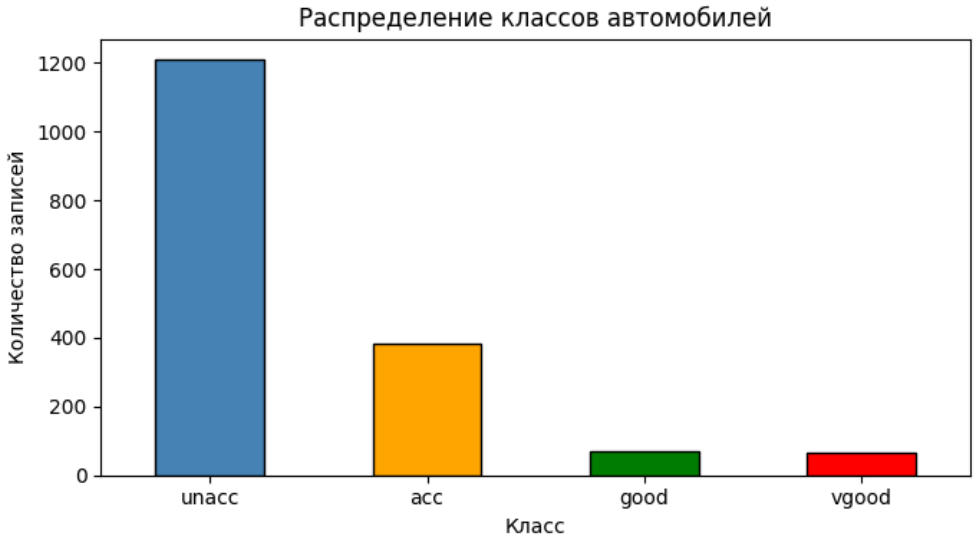

Рисунок 3 – Диаграма распределения классов автомобилей

Для анализа взаимосвязей между признаками была построена тепловая карта корреляций. Категориальные данные временно были преобразованы в числовой формат с помощью LabelEncode для вычисления коэффициентов корреляции.

Листинг 5 – Построение тепловой карты корреляций между признаками

```
data_temp = data.copy()
for col in data_temp.columns:
    data_temp[col] = LabelEncoder().fit_transform(data_temp[col])

plt.figure(figsize=(8, 6))
sns.heatmap(data_temp.corr(), annot=True, fmt='.2f',
cmap='coolwarm')
plt.title('Тепловая карта корреляций')
plt.tight_layout()
plt.show()
```

Анализ тепловой карты (рис. 4) показал, что признак safety (уровень безопасности) имел наибольшую положительную корреляцию с целевым классом автомобиля. Это означает, что более безопасные автомобили, как правило, получали более высокие оценки. Признаки buying (цена покупки) и maint (стоимость обслуживания) показывают умеренную отрицательную корреляцию с классом: чем выше стоимость автомобиля и его обслуживания, тем ниже его оценка. Также была заметна корреляция признака persons (вместимость) с целевой переменной.

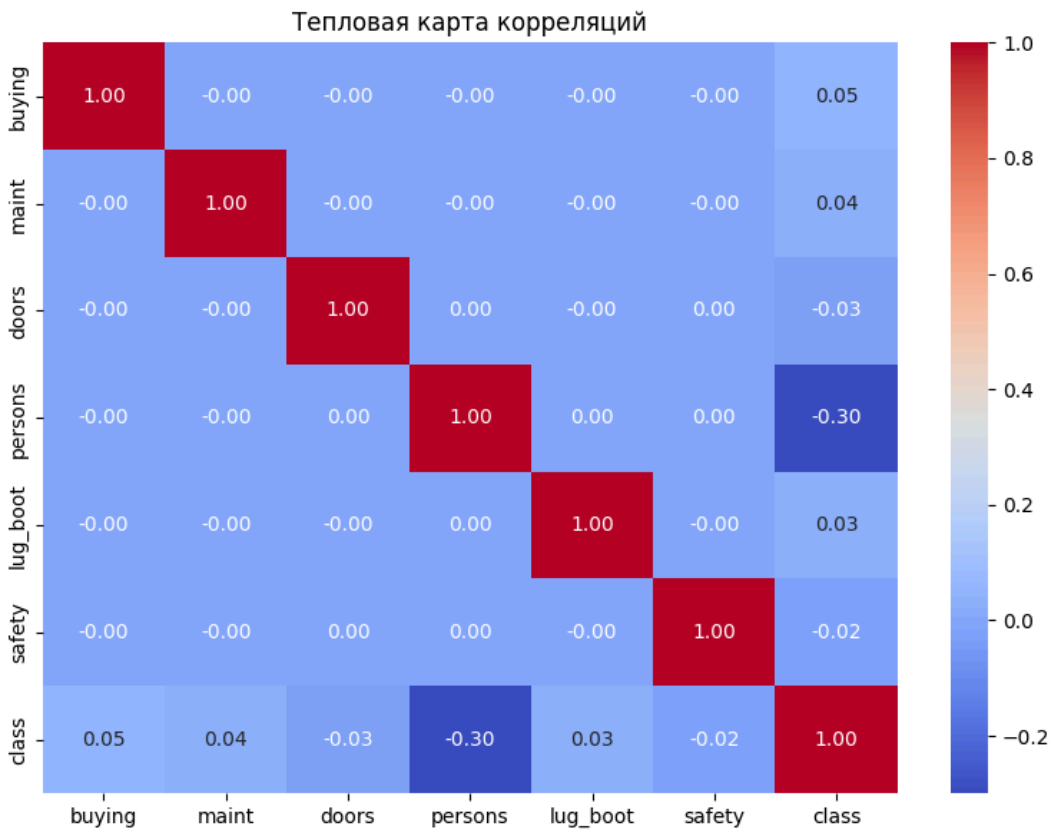

Рисунок 4 – Тепловая карта кореляции

Поскольку нейронные сети работают только с числовыми данными, все категориальные признаки необходимо было преобразовать в числовой формат. Для этого был применён метод кодирования меток (LabelEncoder), который заменял каждое уникальное текстовое значение на соответствующее целое число. Например, для столбца buying значения были преобразованы следующим образом: low стал 0, med — 1, high — 2, а vhigh — 3. Аналогичное преобразование было применено ко всем остальным категориальным признакам.

Листинг 6 – Кодирование категориальных признаков и нормализация данных

```
data_encoded = data.copy()
label_encoders = {}

for col in data_encoded.columns:
    le = LabelEncoder()
    data_encoded[col] = le.fit_transform(data_encoded[col])
    label_encoders[col] = le

X = data_encoded.drop(columns=['class']).values.astype('float32')
y = data_encoded['class'].values.astype('int64')
```

```
scaler = StandardScaler()
X = scaler.fit_transform(X)

NUM_CLASSES = len(np.unique(y))
class_names = label_encoders['class'].classes_
```

После кодирования данные были разделены на матрицу признаков X и вектор y, затем была применена нормализация признаков. Результат показан на рисунке 5.

	buying	maint	doors	persons	lug_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

Рисунок 5 – Преобразованные признаки

Данные были разделены на обучающую (80%) и тестовую (20%) выборки с помощью train_test_split. Параметр stratify=y сохраняет соотношение классов в обеих выборках. Данные были преобразованы в тензоры PyTorch типа float32 для признаков и long для меток классов.

Листинг 7 – Разделение данных и преобразование в тензоры PyTorch

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t  = torch.tensor(X_test,  dtype=torch.float32)
y_test_t  = torch.tensor(y_test,  dtype=torch.long)

print(f'Обучающая выборка: {X_train_t.shape[0]} записей')
print(f'Тестовая выборка:  {X_test_t.shape[0]} записей')
```

Обучающая выборка содержала 1382 записи, тестовая — 346 записей (рис. 6).

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t  = torch.tensor(X_test,  dtype=torch.float32)
y_test_t  = torch.tensor(y_test,  dtype=torch.long)

print(f'Обучающая выборка: {X_train_t.shape[0]} записей')
print(f'Тестовая выборка:  {X_test_t.shape[0]} записей')
✓ 0.0s
```

```
Обучающая выборка: 1382 записей
Тестовая выборка:  346 записей
```

Рисунок 6 – Разделениее на обучающую и тестовую выборки

4.4 Построение архитектуры нейронной сети

Была создана нейронная сеть прямого распространения (FNN) с помощью nn.Sequential. Архитектура включает входной слой, принимающий 6 нормализованных признаков, первый скрытый слой Linear(6 - 64) с функцией активации ReLU, второй скрытый слой Linear(64 - 32) с ReLU и выходной слой Linear(32 - 4) для четырех классов. Функция ReLU вводит нелинейность и помогает избежать затухания градиентов.

Листинг 8 – Создание архитектуры нейронной сети

```
INPUT_SIZE = X_train_t.shape[1]

model = nn.Sequential(
    nn.Linear(INPUT_SIZE, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, NUM_CLASSES)
)

print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f'\nКоличество параметров: {total_params}')
```

Архитектура модели и количество параметров (2660) показаны на рисунке 7.

```python
INPUT_SIZE = X_train_t.shape[1]

model = nn.Sequential(
    nn.Linear(INPUT_SIZE, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, NUM_CLASSES)
)

print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f'\nКоличество параметров: {total_params}')
```

✓ 0.0s

```
Sequential(
  (0): Linear(in_features=6, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=4, bias=True)
)

Количество параметров: 2660
```

Рисунок 7 – Построение модели нейронной сети

4.5 Обучение модели

Перед началом обучения необходимо было задать гиперпараметры, которые управляли процессом оптимизации. Скорость обучения (learning rate) была установлена равной 0.001, что являлось стандартным значением для оптимизатора Adam. Размер батча (batch size) был выбран равным 32, то есть на каждой итерации обучения модель обрабатывала 32 примера одновременно. Количество эпох обучения было установлено в 100, что означало, что модель прошла через весь обучающий датасет сто раз.

В качестве функции потерь использовалась кросс-энтропия (`nn.CrossEntropyLoss`), которая являлась стандартным выбором для задач многоклассовой классификации. Эта функция вычисляла расстояние между предсказанным распределением вероятностей классов и истинными метками. Для оптимизации весов сети был выбран алгоритм Adam (Adaptive Moment Estimation), который автоматически адаптировал скорость обучения для каждого параметра на основе первого и второго моментов градиентов.

На каждой эпохе модель обрабатывала батчи данных, выполняя прямой проход, вычисление потерь, обратное распространение и обновление весов. После каждой эпохи модель тестировалась на тестовой выборке.

Листинг 9 – Цикл обучения модели с оптимизатором Adam

```
LEARNING_RATE = 0.001
BATCH_SIZE    = 32
EPOCHS        = 100

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

train_losses, val_losses = [], []
train_accs,   val_accs   = [], []

for epoch in range(EPOCHS):
    model.train()
    epoch_loss, correct = 0.0, 0

    for i in range(0, len(X_train_t), BATCH_SIZE):
        X_batch = X_train_t[i:i+BATCH_SIZE]
        y_batch = y_train_t[i:i+BATCH_SIZE]

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item() * len(X_batch)
        correct    += (outputs.argmax(1) == y_batch).sum().item()

    train_losses.append(epoch_loss / len(X_train_t))
    train_accs.append(correct / len(X_train_t))

    model.eval()
    with torch.no_grad():
        val_out  = model(X_test_t)
        val_loss = criterion(val_out, y_test_t).item()
        val_acc  = (val_out.argmax(1) ==
y_test_t).float().mean().item()

    val_losses.append(val_loss)
    val_accs.append(val_acc)

print(f'Точность на тестовых данных: {val_accs[-1]:.4f}')
```

Модель достигла точности 99.71% на тестовой выборке (рис. 8).

```python
LEARNING_RATE = 0.001
BATCH_SIZE    = 32
EPOCHS        = 100

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

train_losses, val_losses = [], []
train_accs,   val_accs   = [], []

for epoch in range(EPOCHS):
    model.train()
    epoch_loss, correct = 0.0, 0

    for i in range(0, len(X_train_t), BATCH_SIZE):
        X_batch = X_train_t[i:i+BATCH_SIZE]
        y_batch = y_train_t[i:i+BATCH_SIZE]

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(X_batch)
        correct    += (outputs.argmax(1) == y_batch).sum().item()

    train_losses.append(epoch_loss / len(X_train_t))
    train_accs.append(correct / len(X_train_t))

    model.eval()
    with torch.no_grad():
        val_out  = model(X_test_t)
        val_loss = criterion(val_out, y_test_t).item()
        val_acc  = (val_out.argmax(1) == y_test_t).float().mean().item()

    val_losses.append(val_loss)
    val_accs.append(val_acc)

print(f'Точность на тестовых данных: {val_accs[-1]:.4f}')
```

✓ 5.3s

```
Точность на тестовых данных: 0.9971
```

Рисунок 8 – Обчение модели

4.6 Оценка качества модели

Для анализа процесса обучения были построены графики изменения функции потерь и точности в зависимости от эпохи. Графики позволили оценить качество обучения и выявить возможное переобучение.

Листинг 10 – Визуализация динамики обучения

```python
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].plot(train_losses, label='Training Loss')
axes[0].plot(val_losses,   label='Validation Loss', linestyle='--')
axes[0].set_title('Изменение функции потерь')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('Loss')
```

```
axes[0].legend()

axes[1].plot(train_accs, label='Training Accuracy')
axes[1].plot(val_accs,   label='Validation Accuracy',
linestyle='--')
axes[1].set_title('Изменение точности модели')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('Accuracy')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Графики (рис. 9) показали быстрое снижение потерь в первые 20 эпох и рост точности до 97-99%. Кривые обучающей и тестовой выборок шли близко, что свидетельствовало об отсутствии переобучения.

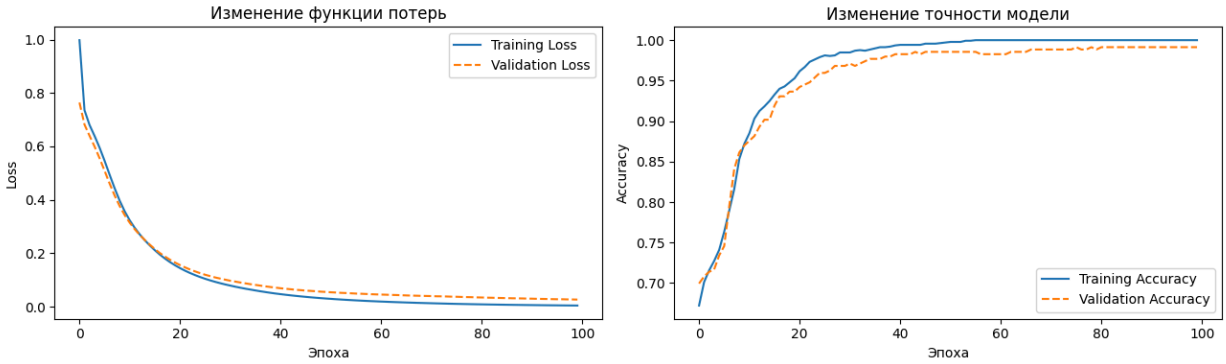

Рисунок 9 – Оценка модели

4.7 Дополнительное исследование: сравнение оптимизаторов

Было проведено сравнение трёх оптимизаторов: AdamW, Adam и Adagrad при фиксированных параметрах (lr=0.001, batch_size=32, epochs=100). Для каждого оптимизатора создавалась новая модель с одинаковой архитектурой. Adam – адаптивный оптимизатор, использующий скользящие средние градиентов. AdamW – модификация Adam с явной регуляризацией весов для снижения переобучения. Adagrad накапливает квадраты градиентов, что приводит к постепенному уменьшению шага обучения.

Листинг 11 – Функция для обучения и сравнения различных оптимизаторов

```
def train_model(optimizer_name):
    torch.manual_seed(42)
    model = nn.Sequential(
        nn.Linear(INPUT_SIZE, 64), nn.ReLU(),
        nn.Linear(64, 32),         nn.ReLU(),
        nn.Linear(32, NUM_CLASSES)
    )

    if optimizer_name == 'Adam':
        optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
    elif optimizer_name == 'AdamW':
        optimizer = optim.AdamW(model.parameters(),
lr=LEARNING_RATE)
    elif optimizer_name == 'Adagrad':
        optimizer = optim.Adagrad(model.parameters(),
lr=LEARNING_RATE)

    criterion = nn.CrossEntropyLoss()
    train_losses, val_losses = [], []
    train_accs,   val_accs   = [], []

    for epoch in range(EPOCHS):
        model.train()
        epoch_loss, correct = 0.0, 0

        for i in range(0, len(X_train_t), BATCH_SIZE):
            X_batch = X_train_t[i:i+BATCH_SIZE]
            y_batch = y_train_t[i:i+BATCH_SIZE]

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item() * len(X_batch)
            correct    += (outputs.argmax(1) ==
y_batch).sum().item()

        train_losses.append(epoch_loss / len(X_train_t))
        train_accs.append(correct / len(X_train_t))

        model.eval()
        with torch.no_grad():
            val_out  = model(X_test_t)
            val_losses.append(criterion(val_out, y_test_t).item())
            val_accs.append((val_out.argmax(1) ==
y_test_t).float().mean().item())

    model.eval()
    with torch.no_grad():
        y_pred = model(X_test_t).argmax(1).numpy()

    final_acc = accuracy_score(y_test, y_pred)
    print(f'[{optimizer_name}] Точность: {final_acc:.4f}')

    return {
        'train_losses': train_losses, 'val_losses': val_losses,
        'train_accs':   train_accs,   'val_accs':   val_accs,
        'final_acc':    final_acc,    'preds':      y_pred
    }

results = {}
```

```
for opt_name in ['AdamW', 'Adam', 'Adagrad']:
    results[opt_name] = train_model(opt_name)
```

Результаты (рис. 10): Adam и AdamW достигли точности 99.1%, Adagrad — только 71.4%. Низкая эффективность Adagrad была связана с накапливаемым уменьшением скорости обучения.

```
Фиксированные параметры: lr=0.001, batch_size=32, epochs=100
--------------------------------------------------------
[AdamW] Точность: 0.9913 | Потери: 0.0261
[Adam] Точность: 0.9913 | Потери: 0.0265
[Adagrad] Точность: 0.7139 | Потери: 0.6225
```

Рисунок 10 – Сравнение оптимизаторов

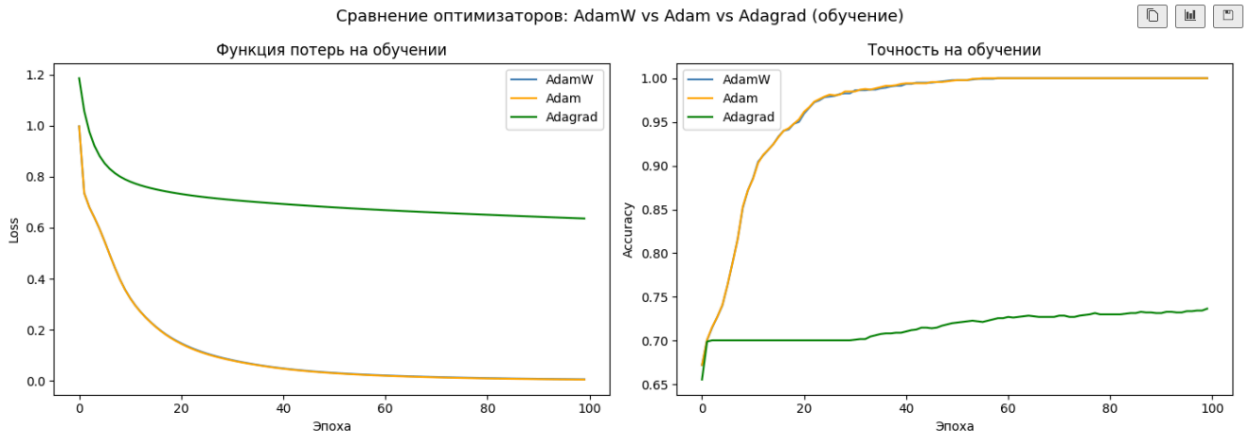

Рисунок 11 – Сравнение оптимизаторов графически

Графики (рис. 11) показали, что Adam и AdamW вели себя схоже: потери быстро снижались, точность росла до высоких значений. Adagrad сходился медленнее и достигал только 74% точности.

5 Выводы

В ходе лабораторной работы была реализована и обучена нейронная сеть прямого распространения для классификации автомобилей на основе датасета Car Evaluation с использованием PyTorch.

Данные были разделены на обучающую (80%, 1382 записи) и тестовую (20%, 346 записей) выборки с сохранением пропорций классов. Архитектура модели включала два скрытых слоя с активацией ReLU и выходной слой на 4 класса.

Базовая модель с оптимизатором Adam достигла точности 99.71% на тестовой выборке. Анализ графиков показал быструю сходимость в первые 20 эпох и стабилизацию без признаков переобучения. В рамках дополнительного исследования были сравнены три оптимизатора: Adam и AdamW показали точность 99.1%, Adagrad — только 71.4%. Низкая эффективность Adagrad была связана с чрезмерным уменьшением скорости обучения.

Таким образом, для данной задачи оптимизаторы Adam и AdamW являлись предпочтительными. AdamW рекомендовался при необходимости защиты от переобучения. Adagrad не подходил из-за агрессивного снижения скорости обучения.

Ссылка на код: «https://colab.research.google.com/drive/1DHSqFIelXyGCp0ip2GJR8egEEfsTr2ZE?usp=sharing»